MACHINE LEARNING · HEALTHCARE ANALYTICS

Six Algorithms, One Dataset, One Honest Answer

Cardiovascular disease prediction from 308,754 survey records: a 10-step guide from the CSV file to a calibrated, threshold-tuned screening model.

Logistic Regression **Naive Bayes** **KNN** **Decision Tree**
Random Forest **AdaBoost**

Python · scikit-learn · seaborn · matplotlib

Md. Anwarul Karim

MS in Data Science and Analytics

Md. Anwarul Karim

THE PROBLEM

Screening for heart disease with survey data

308,754
records after cleaning

16
input features

8.09%
have CVD (Yes)

4,993
positives in the test set

Goal: a low-cost first-stage screen. Catch about 80% of people who have cardiovascular disease, and know exactly what each false alarm costs. CVD is the positive class in every metric below.

The 16 features

- Demographic: Age_Category, Sex
- Health status: General_Health, Checkup, Diabetes, Arthritis, Depression, Skin_Cancer, Other_Cancer
- Lifestyle: Exercise, Smoking_History, Alcohol, Fruit, Green_Vegetables, FriedPotato consumption
- Body: BMI (replaces height and weight)

KNOW THE LIMITS

This is self-reported BRFSS survey data. The model flags people who likely already have the disease. It is a screening aid, not a diagnosis and not a future-risk calculator.

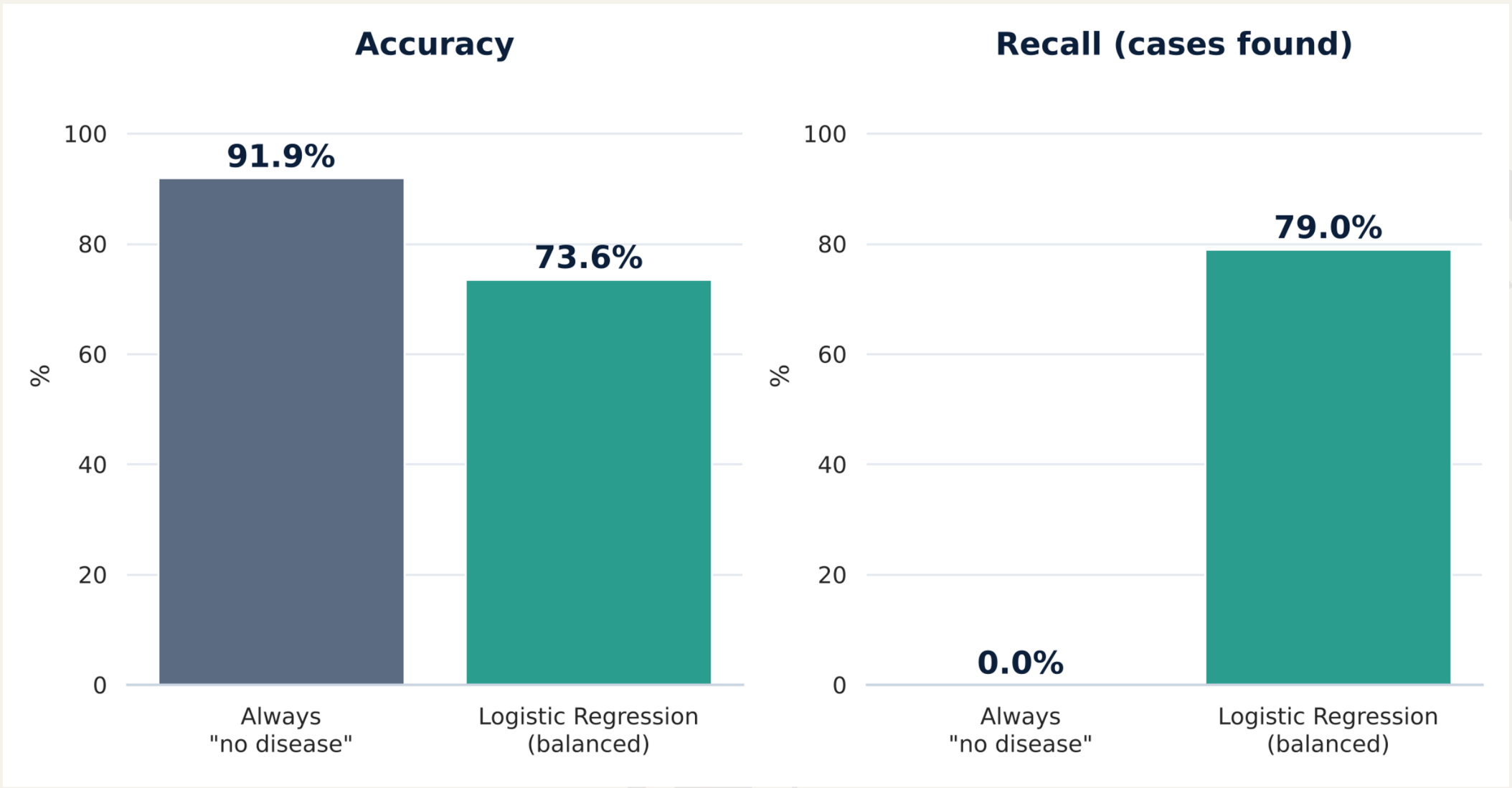
THE TARGET

Heart_Disease: Yes = 1, No = 0. Only about 1 person in 12 is positive, which is why the evaluation matters more than the algorithm.

Md. Anwarul Karim

WHY ACCURACY LIES

A model that ignores every patient scores 91.9%



Predicting “no disease” for all 61,751 test patients is right 91.9% of the time and finds 0 of 4,993 patients. A balanced Logistic Regression has lower accuracy (73.6%) and finds 79.0% of them.

WHAT IT MEANS

With rare outcomes, accuracy rewards ignoring the minority. So every model is judged on recall (cases found), precision, F1, ROC-AUC, PR-AUC (no-skill baseline 0.081), and Brier score (are the probabilities honest?).

প্রায় ৮ জন রোগী হলে, সবাইকে “সুস্থ” বললেই ৯২% accuracy হয়ে যায়।

THE ROADMAP

From CSV file to calibrated threshold in 10 steps

1 Load
read the CSV

2 Clean
impossible BMI only

3 EDA
rates, not counts

4 Encode
true clinical order

5 Split
stratified 80/20

6 Preprocess
scaler inside Pipeline

7 Train
six algorithms, 5-fold OOF

8 Evaluate
CM, ROC, PR at 0.5

9 Calibrate
isotonic + threshold

10 Report
test set, used once

Steps 1–6 prepare the data without leaking test information. Steps 7–8 train and compare six algorithms. Steps 9–10 turn the best one into a screening rule: calibrated probabilities, a threshold chosen on training data only, and one final look at the test set.

Set up the project, read the CSV, remove only impossible values

CODE 1 · Folder layout and how to run

```
cvd_project/
├─ CVD_cleaned.csv        # the 308,854 × 19 survey file
├─ cvd_project.py         # the full 10-step script
└─ figures/               # created automatically

pip install pandas numpy matplotlib seaborn "scikit-learn>=1.2"
python cvd_project.py CVD_cleaned.csv
```

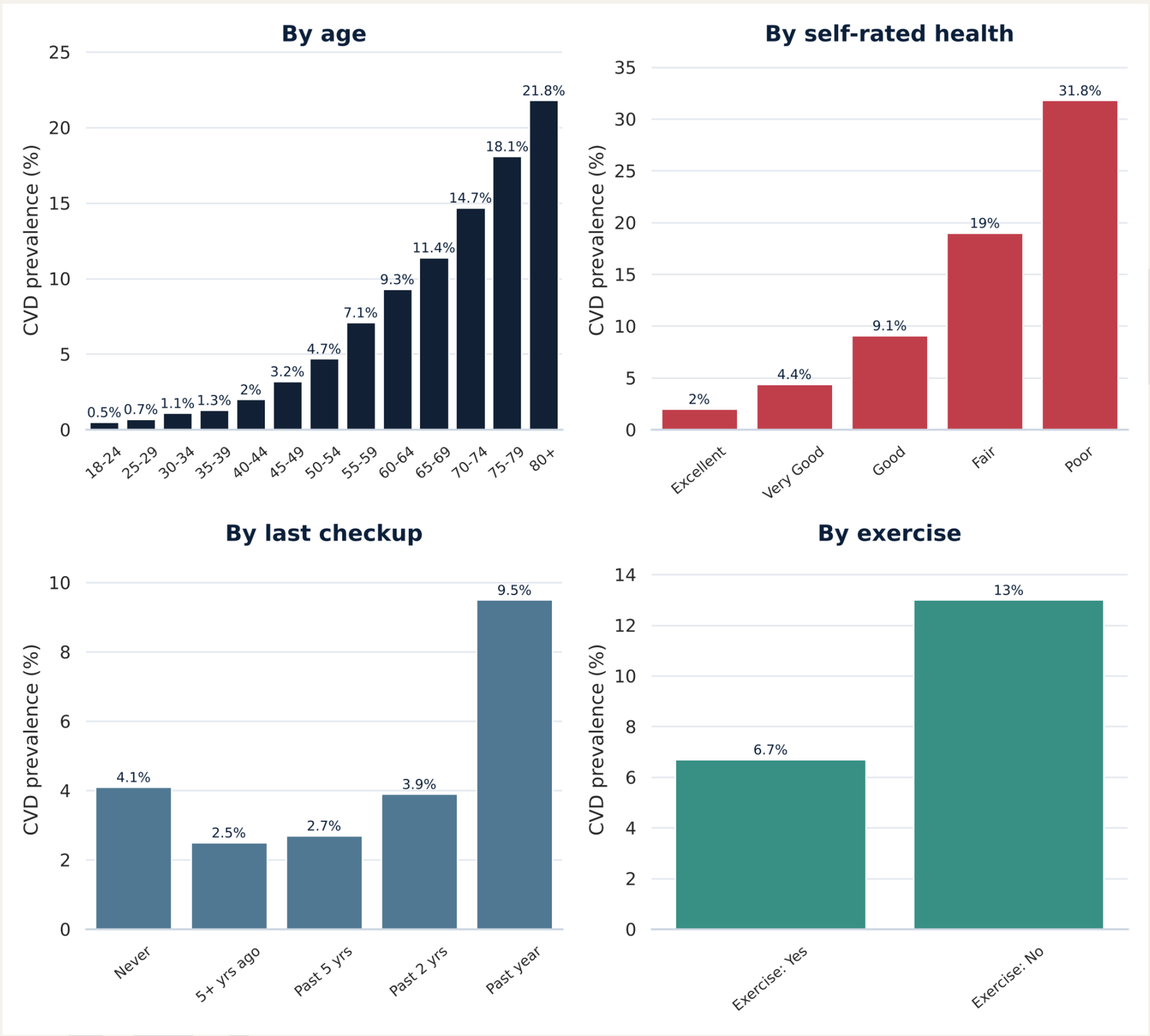
CODE 2 · Load the CSV and clean it

```
df = pd.read_csv("CVD_cleaned.csv")        # (308854, 19)
df = df.drop(columns=["Weight_(kg)", "Height_(cm)"])   # BMI has both
df["Diabetes"] = df["Diabetes"].map({
    "No": "No",
    "No, pre-diabetes or borderline diabetes": "Pre-Diabetes",
    "Yes, but female told only during pregnancy": "Gestational",
    "Yes": "Yes"})
df = df[(df["BMI"] >= 12) & (df["BMI"] <= 75)].copy()
```

- 308,854 rows become 308,754 (99.97% kept). Only 100 rows with impossible BMI are removed.
- 100 duplicate rows are kept: identical survey answers from different people are plausible.
- In Jupyter, delete the line matplotlib.use("Agg") to see plots inline.

Md. Anwarul Karim

Look at rates, not counts



CODE 3 · Prevalence rate per group

```
rate = df["Heart_Disease"].eq("Yes").groupby(df["Age_Category"]).mean() * 100
sns.barplot(x=rate.index, y=rate.values)    # P(CVD | age group)
```

- Age: 0.5% at 18–24 rising to 21.8% at 80+. Self-rated health: 2.0% (Excellent) to 31.8% (Poor).
- The checkup paradox: “checkup in the past year” shows 9.5% versus 4.1% for “never”. Sick people get checked, so this is reverse causation, not a harm of checkups.
- Exercise (13.0% vs 6.7%) is partly an age effect. Do not read any single bar as a cause.

STEPS 4–6 · ENCODE, SPLIT, PREPROCESS

Keep the clinical order and keep the test set clean

CODE 4 · Encode with true clinical order

```
mappings = {
    "General_Health": {"Poor": 0, "Fair": 1, "Good": 2,
                       "Very Good": 3, "Excellent": 4},
    "Age_Category": {a: i for i, a in enumerate(age_order)},
    "Sex": {"Female": 0, "Male": 1},
    "Exercise": {"No": 0, "Yes": 1},   # same for other Yes/No columns
}
for col, mp in mappings.items():
    enc[col] = enc[col].map(mp)
assert enc.isna().sum().sum() == 0      # catches unmapped labels
```

CODE 5 · Stratified 80/20 split, shared by all six models

```
X = enc.drop(columns="Heart_Disease"); y = enc["Heart_Disease"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.20, random_state=42, stratify=y)   # 247,003 / 61,751
```

CODE 6 · Preprocessing that lives inside each model's Pipeline

```
def make_preprocessor(scale_everything=False):
    num = num_cols + ord_cols if scale_everything else num_cols
    ordc = [] if scale_everything else ord_cols
    return ColumnTransformer([
        ("num", StandardScaler(), num),
        ("nom", OneHotEncoder(drop="first", sparse_output=False), nom_cols),
        ("ord", "passthrough", ordc)])
```

- Diabetes has no order, so it is one-hot encoded; the other ordinal columns keep their true order.
- stratify=y keeps 8.1% positives in both parts. The scaler sits inside each Pipeline, so it is learned on training folds only.
- Trees need no scaling. KNN measures distance, so scale_everything=True stops Age_Category (0–12) from swamping BMI.

Logistic Regression and Naive Bayes

Logistic Regression

- Idea: model the log-odds of CVD as a weighted sum of the features, squash it with a sigmoid into a probability, and learn the weights by minimising log-loss.
- class_weight="balanced": positives are only 8%, so this stops the model from ignoring them.
- C=1.0 (default penalty strength): 16 features and 247k rows give little risk of overfitting. max_iter=1000 guarantees convergence.

Naive Bayes (Gaussian)

- Idea: Bayes' theorem plus the “naive” assumption that features are independent once the class is known. Predict the class with the largest P(class) × ∏ P(feature | class).
- var_smoothing=1e-9 (default) adds a tiny variance for numerical stability. No class weights exist, so priors are learned from the data (8.1% CVD).
- Weak spot: age, general health and arthritis are correlated, so the independence assumption is violated.

CODE 7 · Fit Logistic Regression and Naive Bayes

```
# Fit pattern: the same three lines work for every model
pipe = models["Logistic Regression"]      # or models["Naive Bayes"]
pipe.fit(X_train, y_train)                  # learn from training rows only
p = pipe.predict_proba(X_test)[:, 1]        # predicted probability of CVD
```

Md. Anwarul Karim

KNN and Decision Tree

K-Nearest Neighbours (KNN)

- Idea: a patient is classified by a vote of the k most similar training patients (Euclidean distance). Nothing is learned; fit() just stores the data.
- k = 801, chosen by 5-fold cross-validation on training rows only (page 12). Small k is noisy; large k smooths the vote.
- All features are scaled, otherwise Age_Category (0–12) would dominate the distance.

Decision Tree

- Idea: a flow-chart of yes/no questions, each chosen to make the groups purer (lowest Gini impurity). Leaves give the prediction.
- max_depth=8 and min_samples_leaf=50 prune the tree; page 13 shows CV performance falling and training performance rising beyond that, the sign of overfitting.
- class_weight="balanced" gives the rare positives enough influence on each split.

CODE 8 · Fit KNN and the Decision Tree

```
knn = models["KNN"]; knn.fit(X_train, y_train)        # stores the data
dt = models["Decision Tree"]; dt.fit(X_train, y_train)  # grows the tree
p_knn = knn.predict_proba(X_test)[:, 1]    # share of k neighbours with CVD
p_dt = dt.predict_proba(X_test)[:, 1]      # share of CVD in the leaf
```

Md. Anwarul Karim

Random Forest and AdaBoost

Random Forest

- Idea: grow many trees, each on a bootstrap sample of rows and a random subset of features per split, then average their votes. Averaging cuts variance.
- n_estimators=200: page 13 shows no gain after about 100 trees, and extra trees never overfit, so 200 is a safe, cheap choice.
- max_depth=10, min_samples_leaf=25 keep each tree modest; class_weight="balanced" handles the 8% positives.

AdaBoost

- Idea: build depth-1 trees (stumps) one after another; each new stump focuses on the rows earlier stumps got wrong, and the final answer is a weighted vote.
- n_estimators=200 stumps with learning_rate=0.5: shrinking each stump's weight makes boosting steadier on noisy self-reported labels.
- No class_weight option, so it flags few people at 0.5. That is why every model also gets its own threshold (page 18).

CODE 9 · Fit Random Forest and AdaBoost

```
rf = models["Random Forest"]; rf.fit(X_train, y_train)   # 200 trees
ada = models["AdaBoost"]; ada.fit(X_train, y_train)     # 200 stumps
p_rf = rf.predict_proba(X_test)[:, 1]
p_ada = ada.predict_proba(X_test)[:, 1]
```

Md. Anwarul Karim

All six models, same split and preprocessing

CODE 10 · Define the six pipelines

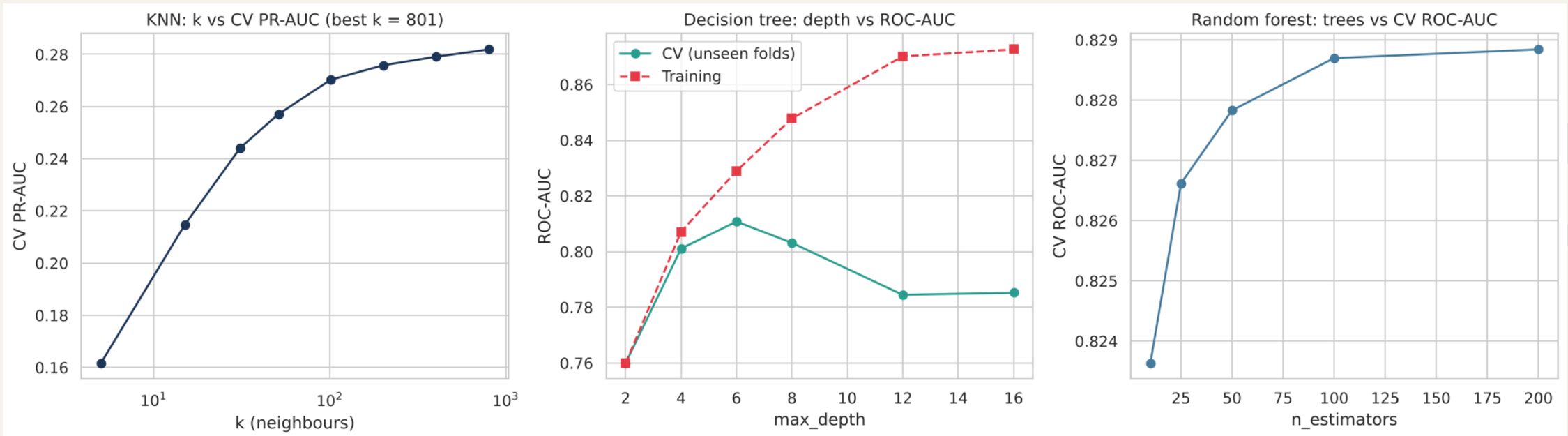
```
models = {
 "Logistic Regression": Pipeline([("prep", make_preprocessor()),
   ("clf", LogisticRegression(class_weight="balanced",
                              max_iter=1000, random_state=42))]),
 "Naive Bayes": Pipeline([("prep", make_preprocessor()),
   ("clf", GaussianNB())]),
 "KNN": Pipeline([("prep", make_preprocessor(scale_everything=True)),
   ("clf", KNeighborsClassifier(n_neighbors=BEST_K, n_jobs=-1))]),   # NEW
 "Decision Tree": Pipeline([("prep", make_preprocessor()),
   ("clf", DecisionTreeClassifier(max_depth=8, min_samples_leaf=50,
                        class_weight="balanced", random_state=42))]),
 "Random Forest": Pipeline([("prep", make_preprocessor()),
   ("clf", RandomForestClassifier(n_estimators=200, max_depth=10,
        min_samples_leaf=25, class_weight="balanced",
        n_jobs=-1, random_state=42))]),
 "AdaBoost": Pipeline([("prep", make_preprocessor()),
   ("clf", AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=1),
        n_estimators=200, learning_rate=0.5, random_state=42))]),
}
```

CODE 11 · Out-of-fold scores on the training set

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
for name, pipe in models.items():
    oof = cross_val_predict(pipe, X_train, y_train, cv=skf,
                            method="predict_proba")[:, 1]   # unseen-fold scores
    print(name, roc_auc_score(y_train, oof), average_precision_score(y_train, oof))
```

Out-of-fold ROC-AUC / PR-AUC (training data only): LR 0.835/0.309; Naive Bayes 0.786/0.218; KNN 0.822/0.286; DT 0.826/0.293; RF 0.835/0.306; AdaBoost 0.835/0.308. These pick the final model, so the test set stays untouched.

Eight values of k, chosen by cross-validation



CODE 12 · KNN: select k using training data only

```
# NEW: compare k values with 5-fold CV on 100,000 training rows (no test data)
for k in [5, 15, 31, 51, 101, 201, 401, 801]:
    pipe_k = Pipeline([("prep", make_preprocessor(scale_everything=True)),
                       ("clf", KNeighborsClassifier(n_neighbors=k, n_jobs=-1))])
    scores = cv_scores(pipe_k, Xk, yk, 5)         # CV ROC-AUC and PR-AUC
BEST_K = knn_tbl["cv_pr"].idxmax()                # highest CV PR-AUC
```

k	CV ROC-AUC	CV PR-AUC
5	0.6815	0.1616
15	0.7597	0.2147
31	0.7909	0.2442
51	0.8035	0.2571
101	0.8136	0.2703
201	0.8168	0.2758
401	0.8177	0.2791
801	**0.8173**	**0.2819**

Very small k is noisy (k=5 gives CV ROC-AUC 0.681); the score climbs as the vote smooths and peaks at k = 801 (CV PR-AUC 0.2819). I picked k by PR-AUC because positives are rare.

Does depth or forest size change performance?

CODE 13 · Tree depth and forest size on 60,000 training rows

```
# NEW: depth grid for the tree, number-of-trees grid for the forest (3-fold CV)
for d in [2, 4, 6, 8, 12, 16]:
    ...  DecisionTreeClassifier(max_depth=d, min_samples_leaf=50, ...)
for n_trees in [10, 25, 50, 100, 200]:
    ...  RandomForestClassifier(n_estimators=n_trees, max_depth=10, ...)
```

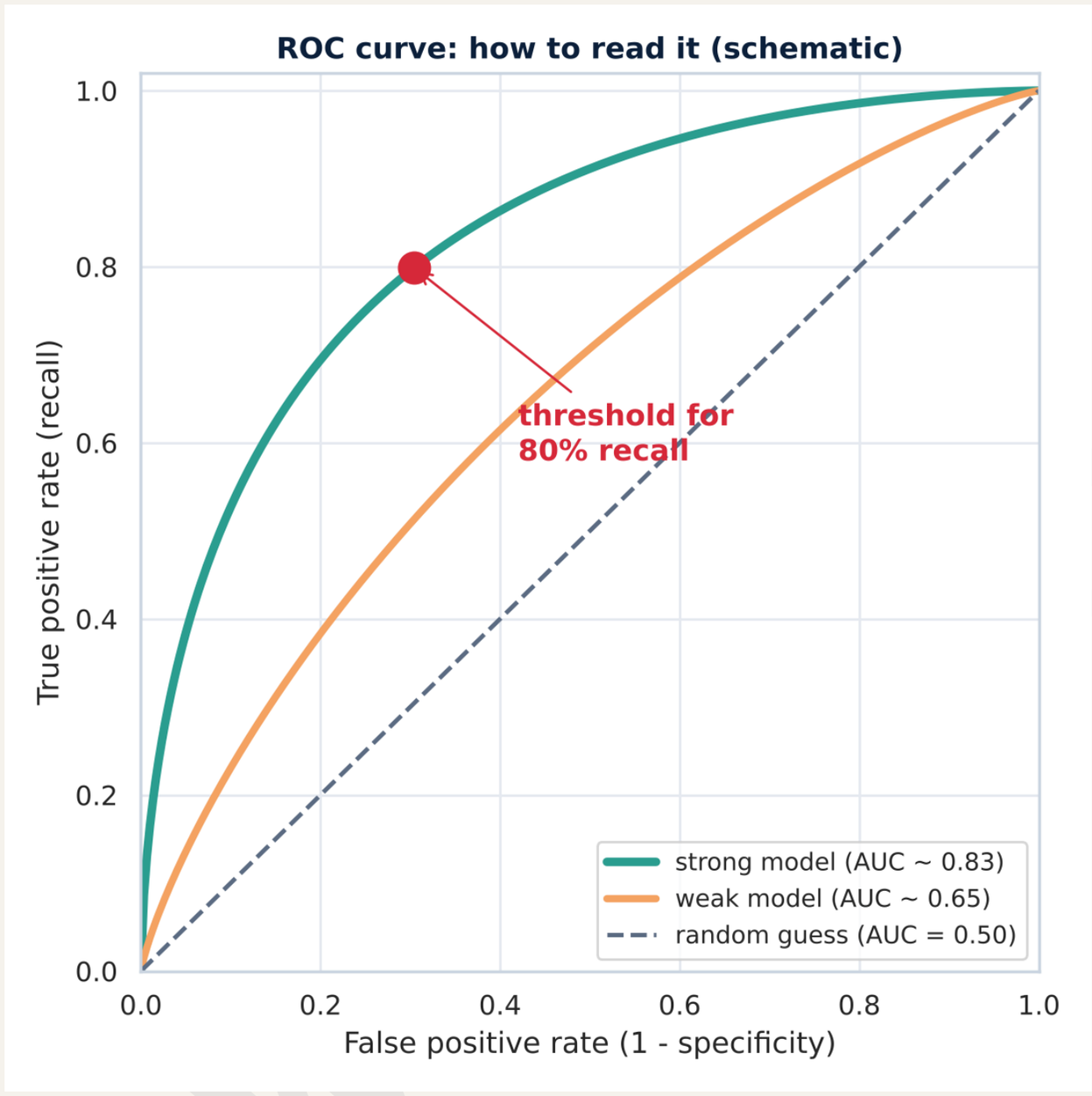
max_depth	CV ROC-AUC	CV PR-AUC	Train ROC-AUC
2	0.7599	0.1720	0.7599
4	0.8011	0.2310	0.8072
6	0.8108	0.2637	0.8289
8	0.8031	0.2682	0.8478
12	0.7845	0.2576	0.8701
16	0.7853	0.2556	0.8726

Decision tree: training ROC-AUC keeps rising (0.760 at depth 2 to 0.873 at depth 16) while cross-validated ROC-AUC peaks at depth 6 (0.8108) and then falls. Deeper trees memorise noise. CV PR-AUC peaks at depth 8, so I keep depth 8.

n_estimators	CV ROC-AUC	CV PR-AUC
10	0.8236	0.2879
25	0.8266	0.2929
50	0.8278	0.2952
100	0.8287	0.2977
200	0.8288	0.2978

Random forest: going from 10 to 200 trees adds 0.0052 ROC-AUC, but only 0.0001 after 100. So the number of trees matters up to about 100 and then plateaus; I keep 200 because extra trees cannot overfit and cost little.

Confusion matrix, ROC and precision-recall



- Recall = TP / (TP + FN): cases found. Specificity = TN / (TN + FP): healthy people cleared.
- Precision = TP / (TP + FP): how many alarms are real. F1 = 2·P·R / (P + R). Accuracy = (TP + TN) / all.
- ROC sweeps the threshold from 1 to 0; AUC is the chance that a random patient with CVD scores above a random patient without. PR-AUC is stricter when positives are rare.

CODE 14 · Metrics for one model

```
tn, fp, fn, tp = confusion_matrix(y_test, p >= 0.5).ravel()
fpr, tpr, thr = roc_curve(y_test, p)      # p = predicted probability
auc = roc_auc_score(y_test, p); ap = average_precision_score(y_test, p)
```

STEP 8 · ONE COMPARISON TABLE

All six models on the same test set (threshold 0.5)

Model	TN	FP	FN	TP	Acc.	Prec.	Recall	F1	AUC
Logistic Reg.	41,490	15,268	1,048	3,945	0.736	0.205	0.790	**0.326**	0.833
Naive Bayes	47,490	9,268	2,466	2,527	0.810	0.214	0.506	0.301	0.788
KNN	56,758	0	4,993	0	**0.919**	0.000	0.000	0.000	0.821
Decision Tree	40,342	16,416	1,023	3,970	0.718	0.195	0.795	0.313	0.825
Random Forest	40,567	16,191	970	4,023	0.722	0.199	**0.806**	0.319	**0.834**
AdaBoost	56,369	389	4,641	352	0.918	**0.475**	0.070	0.123	0.833

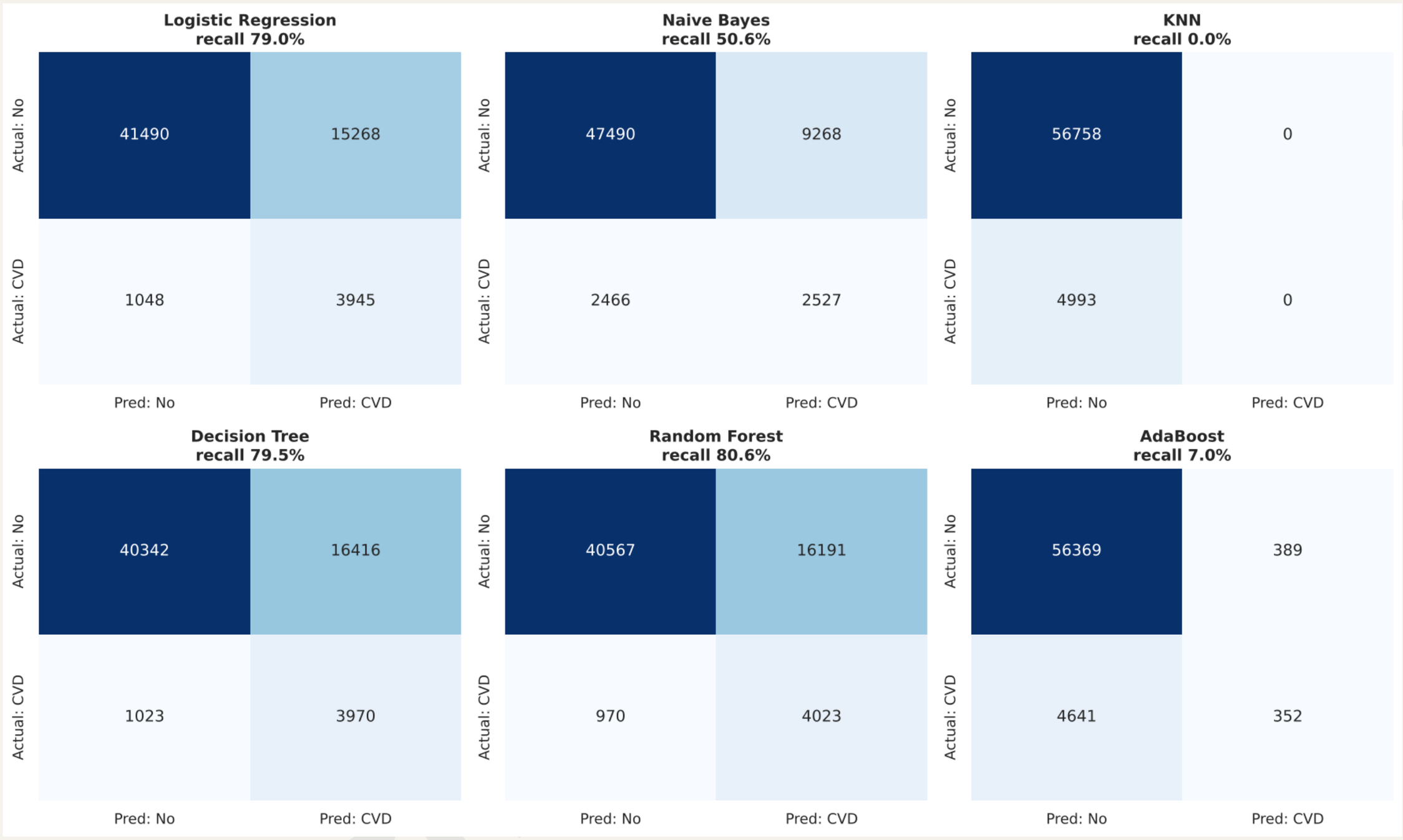
CODE 15 · Evaluate every fitted model on the test set

```
def evaluate(name, y_true, prob, thr):     # one row of the table
    pred = (prob >= thr).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_true, pred).ravel()
    return {"Model": name,
            "Accuracy": accuracy_score(y_true, pred),
            "Precision": precision_score(y_true, pred, zero_division=0),
            "Recall": recall_score(y_true, pred),
            "F1": f1_score(y_true, pred),
            "ROC-AUC": roc_auc_score(y_true, prob),
            "TN": tn, "FP": fp, "FN": fn, "TP": tp}
```

READ IT CAREFULLY

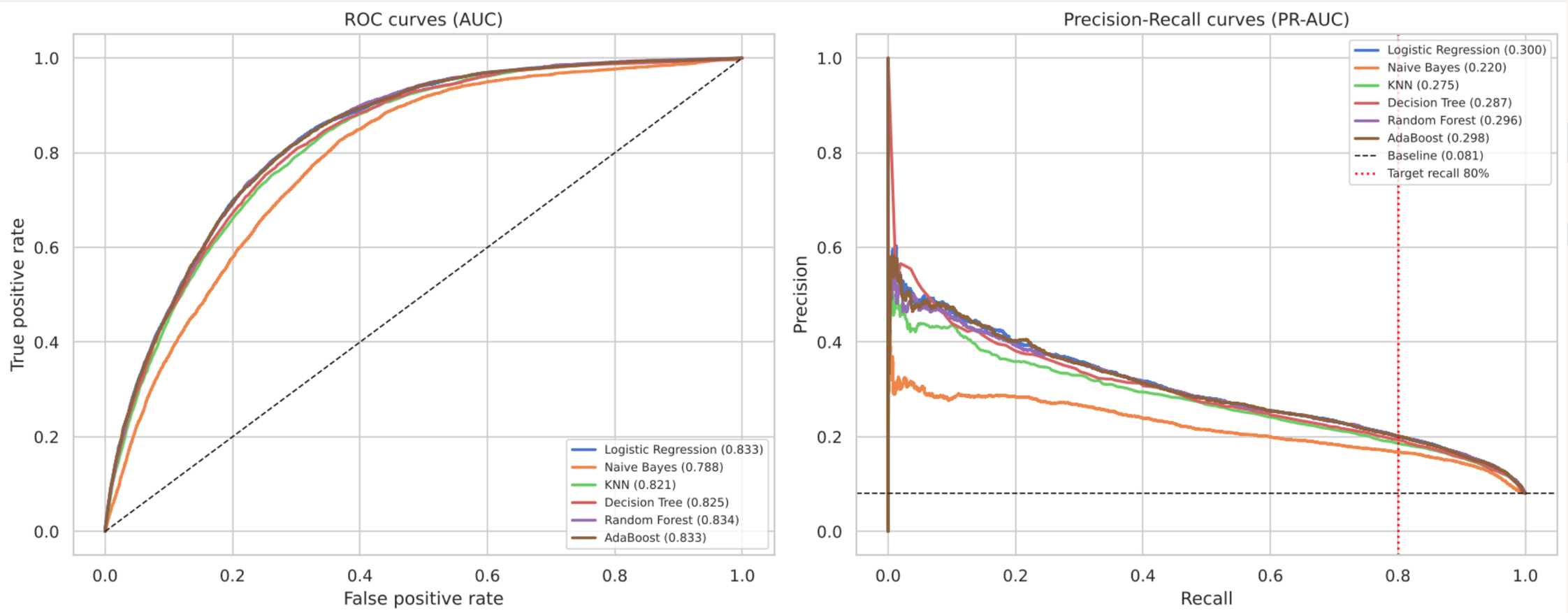
Teal marks the best value per column. KNN has the top accuracy (0.919) but finds only 0.0% of patients: it is almost the “everyone is healthy” model (0.919). AUC is threshold-free, so it is the fair ranking measure here.

Six models at threshold 0.5



- Logistic Regression, Decision Tree and Random Forest catch roughly 79–81% of patients and raise roughly 15,000–16,500 false alarms (class weights at work).
- KNN and AdaBoost flag almost nobody at 0.5, so their recall is near zero: they have no class weights, and 0.5 is the wrong cut-off for an 8%-positive problem.
- Naive Bayes sits in between (recall 50.6%), because its probabilities are overconfident.

The ranking quality of each model, threshold-free



- ROC-AUC: Random Forest 0.834, Logistic Regression 0.833, AdaBoost 0.833, Decision Tree 0.825, KNN 0.821, Naive Bayes 0.788.
- Random Forest, Logistic Regression and AdaBoost are within 0.0009 of each other, far inside the ±0.005 sampling noise of this test set. Decision Tree, KNN and Naive Bayes rank patients clearly worse.
- PR curves tell the same story: all curves sit well above the 0.081 no-skill line, and none reaches high precision at 80% recall.

WHAT IT MEANS

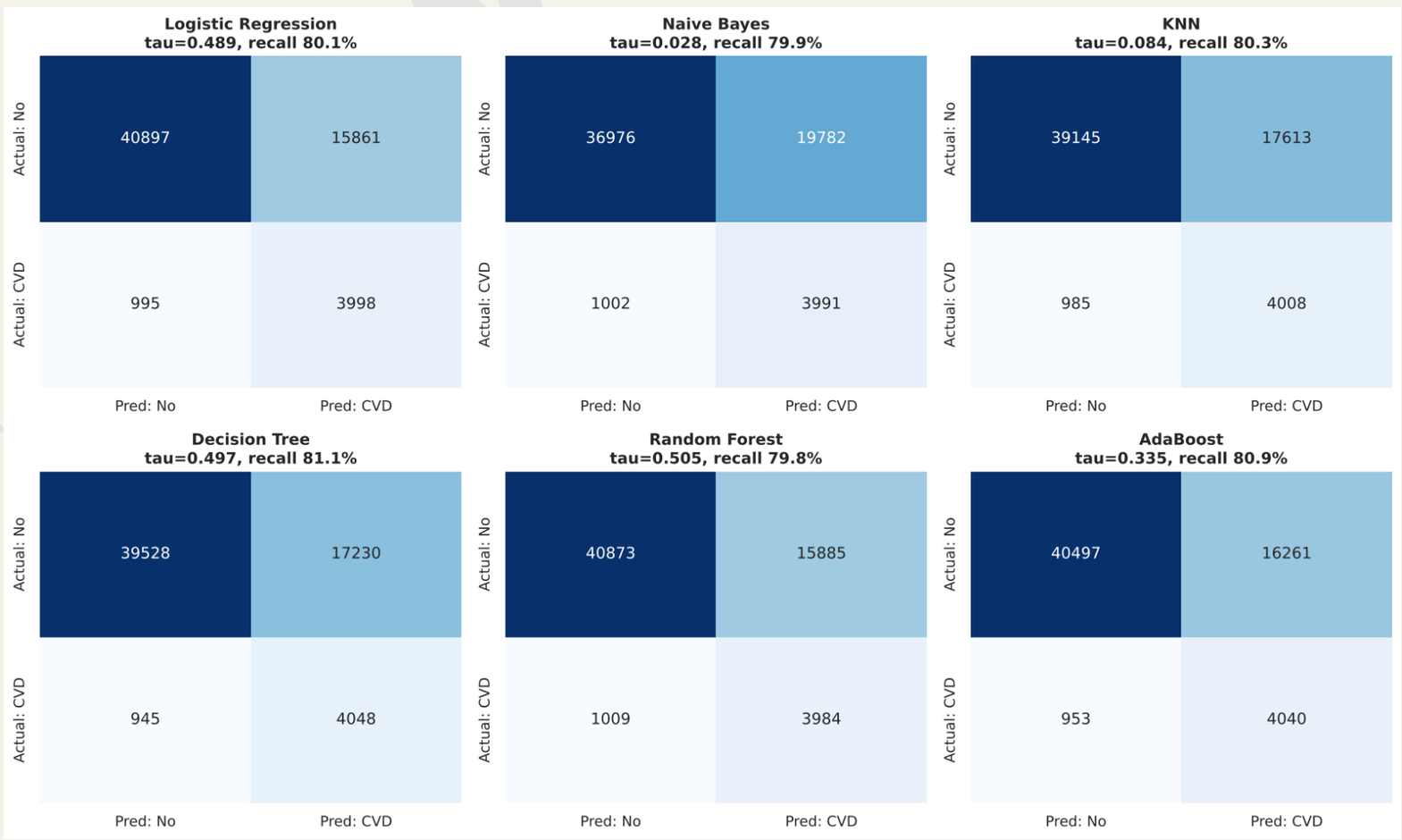
The six algorithms differ by at most 0.046 AUC, and the top three are indistinguishable. The choice among them should rest on simplicity, calibration and cost, not on the third decimal.

Every model at its own 80%-recall threshold

CODE 16 · Matched-recall threshold per model

```python
# NEW: threshold for 80% recall, found on training out-of-fold scores only
def tau_for_recall(y_true, prob, target=0.80):
    _, rec, thr = precision_recall_curve(y_true, prob)
    return thr[np.where(rec[:-1] >= target)[0][-1]]
```

Model	τ	TN	FP	FN	TP	Acc.	Prec.	Recall	F1	AUC
Logistic Reg.	0.489	40,900	15,858	995	3,998	**0.727**	**0.201**	0.801	**0.322**	0.833
Naive Bayes	0.028	36,980	19,778	1,002	3,991	0.663	0.168	0.799	0.278	0.788
KNN	0.084	39,145	17,613	985	4,008	0.699	0.185	0.803	0.301	0.821
Decision Tree	0.497	39,528	17,230	945	4,048	0.706	0.190	**0.811**	0.308	0.825
Random Forest	0.505	40,873	15,885	1,009	3,984	0.726	0.201	0.798	0.321	**0.834**
AdaBoost	0.335	40,497	16,261	953	4,040	0.721	0.199	0.809	0.319	0.833

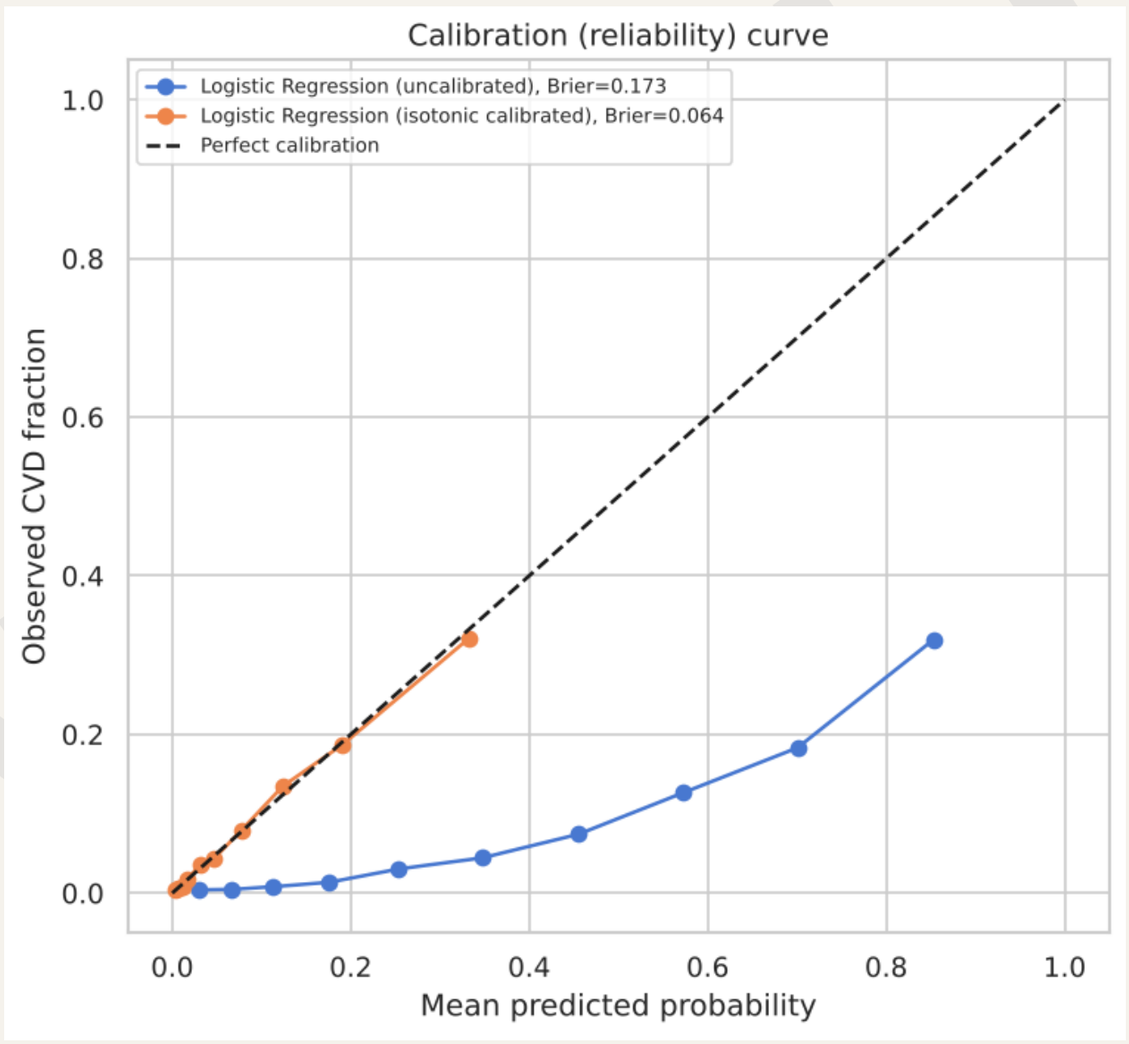


With recall held near 80%, accuracy and precision become comparable. Logistic Regression has the best accuracy (0.727) and F1 (0.322); Random Forest and AdaBoost are within about 0.01.

Make probabilities honest, then choose the cut-off

CODE 17 · Isotonic calibration and the 80%-recall threshold

```
best = oof_table["OOF PR-AUC"].idxmax()        # chosen on training data only
oof_raw = oof_probs[best]                       # out-of-fold scores (step 7)
oof_cal = np.zeros_like(oof_raw)                # NEW: honest calibrated scores
for tr, va in skf.split(oof_raw.reshape(-1, 1), y_train):
    iso_cv = IsotonicRegression(out_of_bounds="clip").fit(oof_raw[tr], y_train.iloc[tr])
    oof_cal[va] = iso_cv.predict(oof_raw[va])   # NEW
iso = IsotonicRegression(out_of_bounds="clip").fit(oof_raw, y_train)  # NEW
tau = tau_for_recall(y_train, oof_cal)          # highest tau with recall >= 80%
p_final = iso.predict(models[best].predict_proba(X_test)[:, 1])
```



- Best model by training out-of-fold PR-AUC: Logistic Regression (0.3094).
- Threshold τ* = 0.0884. The test set never influenced it.
- Brier score: 0.1726 before calibration → 0.0641 after. A predicted risk now matches the observed rate. (class_weight="balanced" had inflated the raw probabilities.)

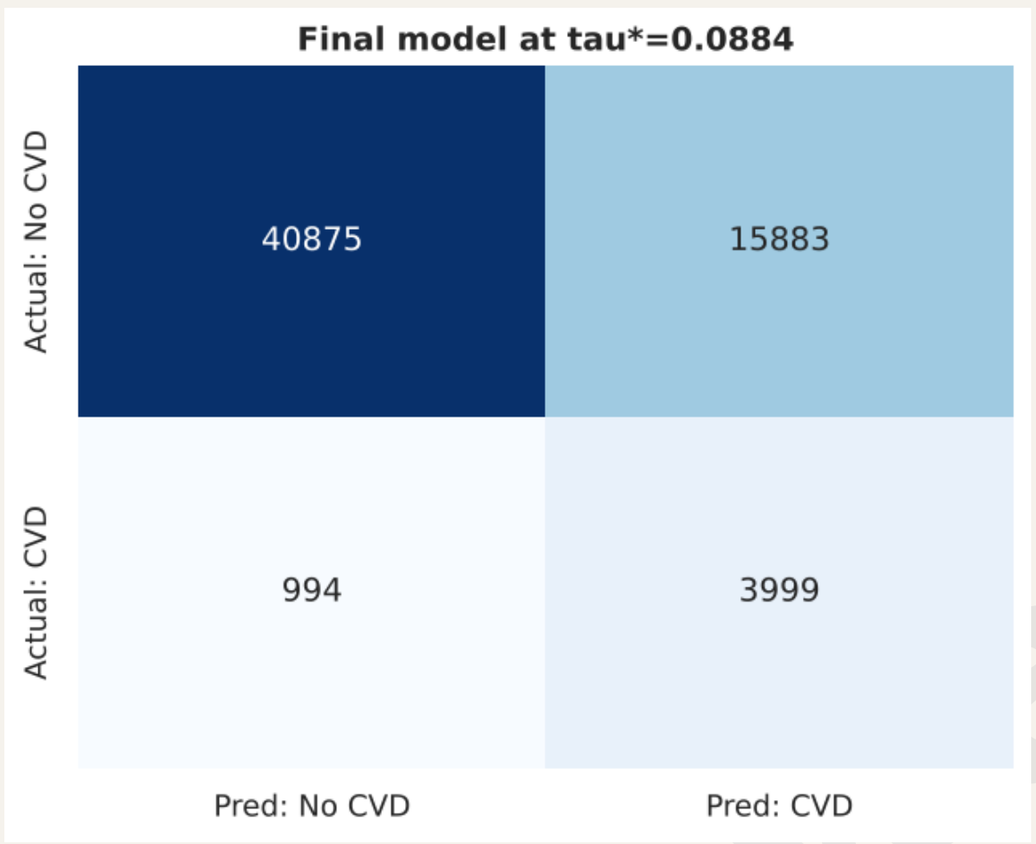
Use the test set once, at the pre-chosen threshold

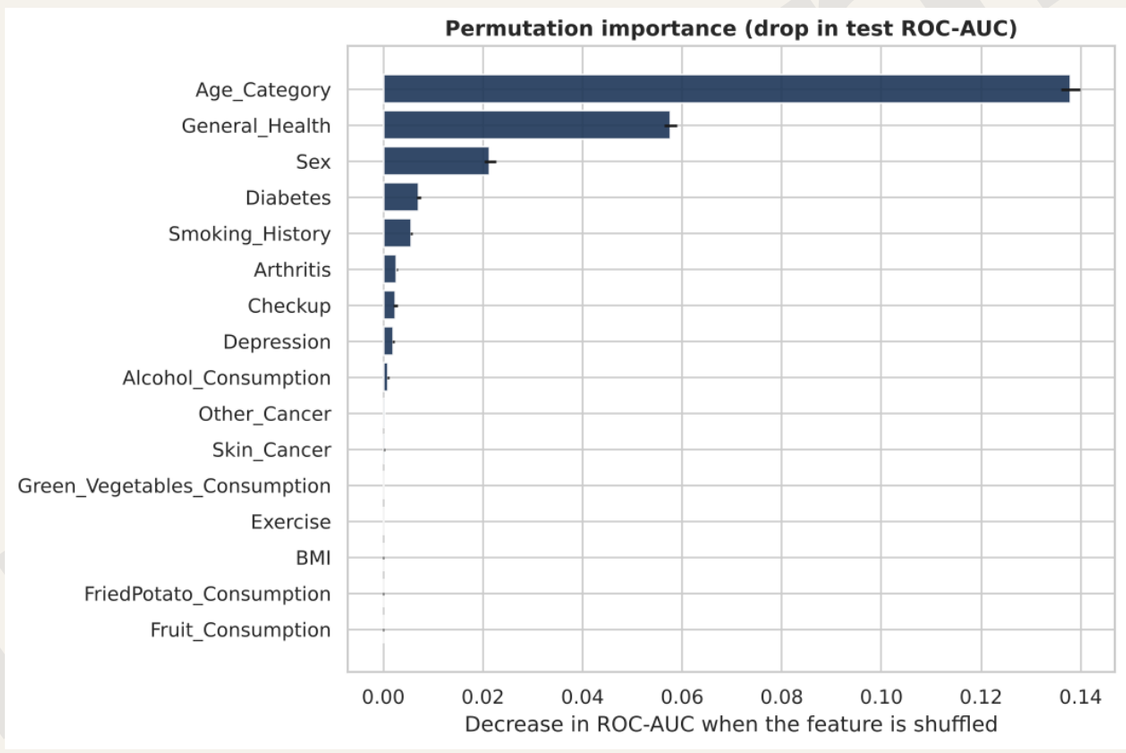
0.833
ROC-AUC, 95% CI 0.828–0.838

0.295
PR-AUC (no-skill 0.081)

80.1%
Recall, CI 79.0–81.3%

72.0%
Specificity





CODE 18 · Final test report, bootstrap intervals and importance

```
pred = (p_final >= tau).astype(int)
tn, fp, fn, tp = confusion_matrix(y_test, pred).ravel()
rng = np.random.default_rng(42)              # NEW: 500 bootstrap resamples
for _ in range(500):
    i = rng.integers(0, len(y_test), len(y_test))   # then recompute the metrics
perm = permutation_importance(models[best], X_test, y_test,   # NEW
                              scoring="roc_auc", n_repeats=5)
```

- At τ* = 0.0884: 3,999 cases caught, 994 missed, 15,883 false alarms, 40,875 correctly cleared. Precision 20.1%: about 1 in 5 flagged people has CVD.
- Biggest drivers (drop in ROC-AUC when shuffled): Age_Category 0.138, General_Health 0.058, Sex 0.021. Lifestyle columns add little beyond these.

Answers, based on the numbers above

- Highest test accuracy: KNN (0.919) at threshold 0.5, with recall of only 0.0%. At the matched 80% recall, Logistic Regression is highest (0.727).
- Highest recall for CVD: Random Forest (80.6%) at 0.5. Other models are close behind except KNN, AdaBoost and Naive Bayes. At matched recall every model sits near 80% by design.
- Precision, recall, F1 and ROC-AUC: Random Forest, Logistic Regression and AdaBoost are essentially tied (AUC 0.833–0.834; matched-recall precision about 0.20, F1 about 0.32). Decision Tree and KNN trail; Naive Bayes is clearly worst (AUC 0.788).

RECOMMENDATION: LOGISTIC REGRESSION, CALIBRATED, WITH T* = 0.0884

It ties the best ranking models within noise, has the highest training out-of-fold PR-AUC, is the simplest to explain and refit, and calibrates cleanly (Brier 0.173 → 0.064). At this threshold it finds 80.1% of CVD cases.

WHY RECALL COMES FIRST

A false negative is a person with heart disease told “low risk”: no follow-up, a delayed diagnosis, possibly a preventable event. A false positive is a healthy person sent for a confirmatory check: some cost and worry, but recoverable. So the screen accepts low precision (about 20%) to keep recall near 80%.

Random Forest is a fair alternative if you ever need interactions captured automatically, but here it adds cost and complexity for no measurable gain.

What this model can and cannot do

Why accuracy alone is not enough for a medical classifier

- Only 8.1% of people have CVD, so “everyone is healthy” scores 91.9% accuracy while finding nobody.
- Accuracy counts a missed patient and a false alarm as equally bad. In screening they are not.
- It depends on the threshold and the prevalence. Recall, precision, F1, ROC-AUC and calibration show what accuracy hides.

Limitations

- Self-reported, cross-sectional survey data: the model detects existing disease, not future events, and answers carry recall bias.
- One random split, one dataset, no external validation. The file has no survey weights, so results describe this sample, not the US population.
- The 80% recall target is a policy choice, and precision near 20% means a flag is a prompt for follow-up, not a diagnosis.

Reproduce it

CODE 19 · Run everything

```
pip install pandas numpy matplotlib seaborn "scikit-learn>=1.2"
python cvd_project.py CVD_cleaned.csv    # results land in ./figures
```

Md. Anwarul Karim
MS in Data Science and Analytics

Md. Anwarul Karim